

אילה — התחברות ודאשבורד

שכבת האבטחה, ניהול משתמשים ודאשבורד מנהלת

מפתחת 1 · Infrastructure & Admin

1. פתיחה — מה לומר (30 שניות)

"שלום, אני אילה. אחראית על שכבת האבטחה והדאשבורד של WigFlow — מערכת ניהול סלון הפאות. בניתי את מערכת ההתחברות, ניהול המשתמשים והדאשבורד שמאפשר למנהלת לראות את כל הפאות והתיקונים במקום אחד, לנהל את הצוות ולעקוב אחרי עומס העובדות."

2. מה עשיתי

- מערכת התחברות (Login) — שם משתמש וסיסמה, אימות מאובטח, והפניה אוטומטית לפי תפקיד.
- אבטחה (JWT + bcrypt) — Token לכל בקשה, סיסמאות מוצפנות, Middleware להרשאות.
- ניהול צוות (Team Management) — הוספה, עריכה, הקפאה ומחיקה של עובדות.
- דאשבורד מנהלת — טבלת סקירה מאוחדת של פאות חדשות + תיקונים.
- עומס עובדות (Workers Load) — כרטיסיות עם מספר משימות פעילות לכל עובדת.
- פעולות מנהלת — סימון "נמסר", מחיקה עם קוד אישור, ניהול דחיפות.
- ניווט חכם — תפריט שונה לכל תפקיד (מנהלת / עובדת / בקרת איכות).

3. קבצים מרכזיים

שכבה	קובץ	תפקיד
Client	components/Auth/LoginForm/LoginForm.tsx	מסך התחברות + ניתוב חכם
Client	components/Dashboard/TeamManagement/TeamManagement.tsx	ניהול צוות
Client	components/Dashboard/MainOverviewTable/MainOverviewTable.tsx	טבלת סקירה כללית
Client	components/Dashboard/WorkersLoadStatus/WorkersLoadStatus.tsx	עומס עובדות
Client	axiosConfig.ts	שליחת Token בכל בקשה
Server	Middlewares/authMiddleware.ts	verifyToken, verifyAdmin, verifyWorker, verifyQC
Server	Models_Service/User/userService.ts	loginUser, createUser, bcrypt + JWT
Server	Routers/userRouter.ts	login, CRUD, — API משתמשים — workload
Tests	tests/admin_infra.test.ts	login, RBAC, dashboard API בדיקות

4. הסבר על קטעי קוד

קטע 1 — התחברות וניתוב חכם (Client)

קובץ: LoginForm.tsx — שורות 20–45

```
try {
  const response = await axios.post('/users/login', {
    username, password
  });

  const { token, user } = response.data;
  localStorage.setItem('token', token);
  localStorage.setItem('user', JSON.stringify(user));
  axios.defaults.headers.common['Authorization'] = `Bearer ${token}`;

  // ניתוב חכם אחרי ההתחברות
  if (user.role === 'Admin') {
    window.location.href = '/';
  } else if (user.role === 'Worker') {
    if (user.specialty?.includes('בקר') || user.specialty?.includes('איכות')) {
      window.location.href = '/qa';
    } else {
      window.location.href = '/repairs/tasks';
    }
  } else if (user.role === 'QC' || user.role === 'Inspector') {
    window.location.href = '/qa';
  }
}
```

מה קורה כאן:

- המשתמשת שולחת שם משתמש וסיסמה לשרת (POST /api/users/login).
- השרת מחזיר **Token** (מפתח אבטחה) ופרטי משתמש.
- ה-Token נשמר ב-localStorage ונשלח בכל בקשה הבאה דרך Axios.
- **ניתוב חכם:** מנהלת → דף ראשי, עובדת רגילה → משימות, בקרת איכות → /qa.

קטע 2 — Middleware לאבטחה (Server)

קובץ: authMiddleware.ts — שורות 11–35

```
export const verifyToken = (req, res, next) => {
  const token = req.header('Authorization')?.replace('Bearer ', '');
  if (!token) {
    return res.status(401).json({ message: 'נדרשת אבטחה' });
  }
  try {
    const decoded = jwt.verify(token, SECRET_KEY);
  }
}
```

```

    req.user = decoded;
    next();
  } catch (error) {
    res.status(401).json({ message: 'טוקן לא תקין או פג תוקף' });
  }
};

export const verifyAdmin = (req, res, next) => {
  verifyToken(req, res, () => {
    if (req.user && req.user.role === 'Admin') {
      next();
    } else {
      res.status(403).json({ message: 'גישת חסומה: נדרשת הרשאת מנהלת' });
    }
  });
};

```

מה קורה כאן:

- `verifyToken` — בודק שיש `Token` תקין בכל בקשה לשרת. אם אין — שגיאה 401.
- `verifyAdmin` — מאפשר גישה רק למנהלת. עובדת רגילה תקבל 403.
- זה מגן על נתונים רגישים — רק מי שמורשה יכול לראות את הדאטאבז.

קטע 3 — הנפקת JWT בשרת (Server)

קובץ: `userService.ts` — פונקציית `loginUser`

```

//שוראת סיסמה מוצפנת
const isMatch = await bcrypt.compare(password, user.password);
if (!isMatch) throw new AppError('401', 'שם משתמש או סיסמה שגויים');

// חסימת חשבון מוקפא
if (user.isActive === false) {
  throw new AppError('403', 'פנה למנהלת');
}

// הנפקת Token
const token = jwt.sign(
  { id: user._id, role: user.role },
  SECRET_KEY,
  { expiresIn: '1d' }
);

```

מה קורה כאן:

- הסיסמה לא נשמרת בטקסט גלוי — היא מוצפנת עם **bcrypt**.
- אם החשבון מוקפא (`isActive: false`) — ההתחברות נחסמת.
- ה-`Token` מכיל את ה-ID והתפקיד של המשתמש, ותקף ליום אחד.

5. תסריט הדגמה

1 פתחי את האפליקציה → מסך Login. הזיני: admin / password123 .

2 הסבירי: "אחרי Login, המערכת מפנה אותי לדף הראשי כי אני מנהלת."

3 עברי ל- dashboard/ — הציגי את **טבלת הסקירה** (פאות + תיקונים).

4 הציגי **עומס עובדות** — כמה משימות פעילות לכל עובדת.

5 ב- **ניהול צוות** — הוסיפי עובדת חדשה (שם, תפקיד, התמחות).

6 הקפיאי חשבון עובדת → נסי להתחבר כעובדת הזו (תקבלי הודעת שגיאה).

7 סמני פאה כ **"נמסרה"** מהטבלה — הסבירי שהסטטוס מתעדכן ב-DB.

6. טכנולוגיות

Express Middleware

bcryptjs

JWT (jsonwebtoken)

Axios

TypeScript

React 18

Jest + Supertest

MongoDB / Mongoose

7. שאלות ותשובות למורה

ש: איך JWT עובד במערכת שלך?

כששתמשת מתחברת, השרת בודק את הסיסמה עם bcrypt ומנפיק Token (JWT) שמכיל את ה-ID והתפקיד. ה-Token נשמר ב-localStorage ונשלח בכל בקשה ב-Header: Authorization: Bearer <token>. Middleware בשרת (verifyToken) בודק את ה-Token לפני כל פעולה רגישה. ה-Token תקף ליום אחד.

ש: מה קורה כשעובדת מוקפאת?

המנהלת מקפאה חשבון דרך Team Management (שדה isActive: false). בפונקציית loginUser בשרת יש בדיקה: אם isActive === false — השרת מחזיר שגיאה 403 עם הודעה "החשבון הוקפא". העובדת לא יכולה להתחבר עד שהמנהלת מפעילה אותה מחדש.

ש: איך הדאשבורד מאחד פאות חדשות ותיקונים?

MainOverviewTable מבצע שני קריאות API במקביל: GET /api/wigs (פאות חדשות) ו-GET /api/repairs/dashboard-view (תיקונים). הוא מאחד את שני המערכים לטבלה אחת עם עמודות: קוד פאה, לקוחה, סטטוס, תחנה נוכחית, עובדת משובצת. כך המנהלת רואה הכל במקום אחד.

ש: מה ההבדל בין verifyToken ל-verifyAdmin?

verifyToken בודק רק שהשתמשת מחוברת (יש Token תקין) — מתאים לכל משתמשת מחוברת. verifyAdmin בודק גם Token וגם שהתפקיד הוא Admin — מתאים לפעולות מנהלת בלבד (צפייה בכל המשתמשים, מחיקת פאות, ניהול צוות).

ש: למה השתמשת ב-localStorage ולא ב-Session?

JWT הוא Stateless — השרת לא שומר מידע על ההתחברות. ה-Token עצמו מכיל את כל המידע הנדרש. localStorage שומר את ה-Token בדפדפן כך שהשתמשת נשארת מחוברת גם אחרי רענון הדף. Axios (axiosConfig.ts) Interceptor מצרף את ה-Token אוטומטית לכל בקשה.

ש: איך עובד הניתוב החכם אחרי Login?

ב-`LoginForm.tsx`, אחרי קבלת ה-Token, הקוד בודק את `user.role` ו-`user.specialty: Admin` → כך
Worker, / רגילה → `Worker/`, `repairs/tasks` עם התמחות בקרה → `/qa/` → `QC/Inspector`, `qa`.
כל משתמשת מגיעה ישירות למסך הרלוונטי לה.

ש: איך מחשבים את עומס העובדות?

`WorkersLoadStatus` שולף את כל העובדות מ-`/api/users` GET, ואז סופר כמה פאות חדשות
ותיקונים משובצים לכל עובדת (לפי `assignedWorkers` בפאות ו-`assignedTo` במשימות תיקון).
התוצאה מוצגת בכרטיסיות ויזואליות.

ש: אילו בדיקות כתבת?

ב-`admin_infra.test.ts`: בדיקה ש-Login מחזיר Token, שעובדת רגילה מקבלת 403 על נתיבי
Admin, ו-`/api/repairs/dashboard-view` GET מחזיר מערך תקין. הבדיקות רצות ב-CI עם Jest +
Supertest.